



EcoQuest

Plataforma de concienciación ambiental gamificada

Equipo Prompt · IES Doctor Balmis · DAM 2025–2026

App Android · API REST · Panel WPF

Introducción

Por qué EcoQuest

- La gente quiere actuar, pero no sabe cómo
- Falta de canales para organizarse localmente
- Las apps verdes existentes no enganchan

Nuestra propuesta

- Comunidades vecinales conectadas
- Eventos y retos ecológicos reales
- Sistema de puntos que motiva

Objetivo

Fomentar hábitos sostenibles mediante comunidades locales, eventos reales y recompensas.

Usuarios

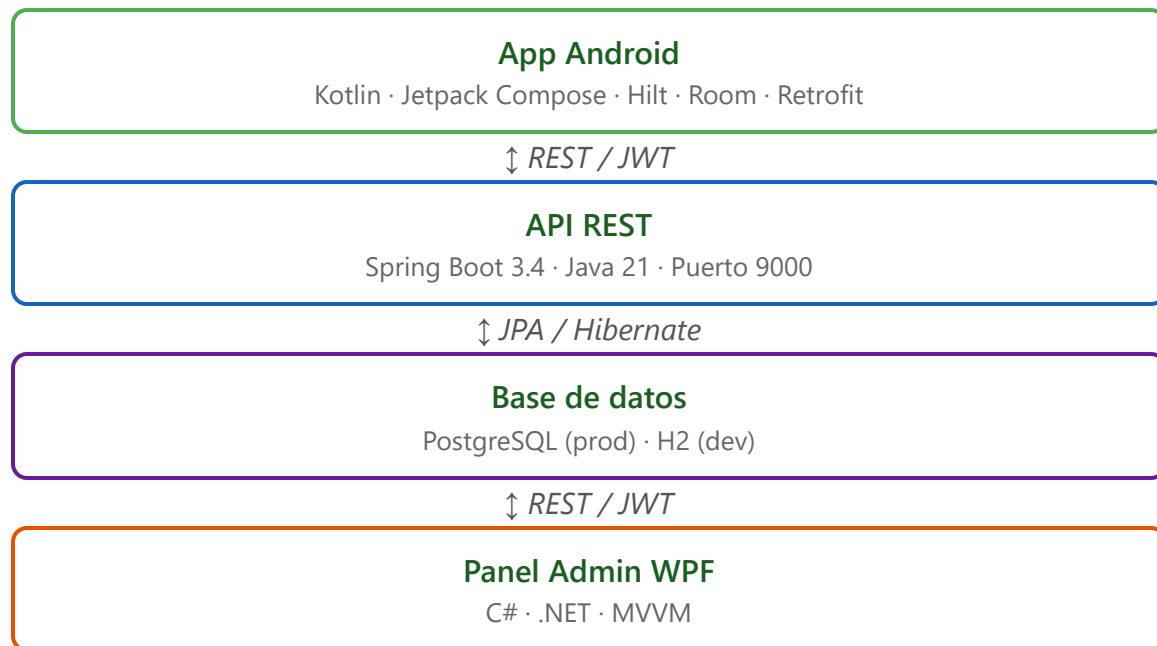
Usuario App Android

Administrador Panel WPF

Integración total

Móvil y escritorio comparten datos en tiempo real a través de la misma API REST.

Arquitectura del sistema



Android — MVVM por pantalla

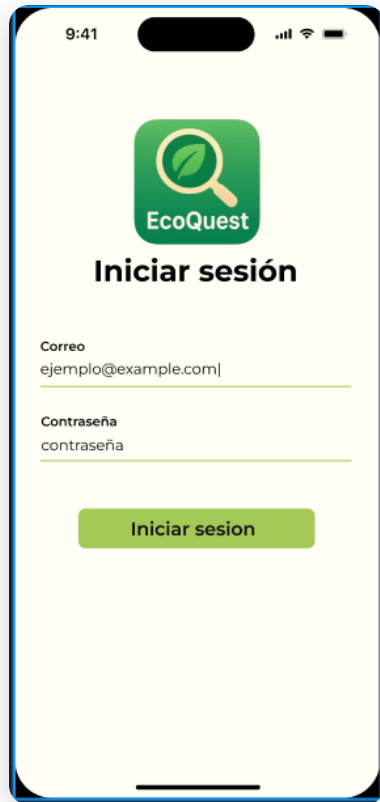
```
Compose UI ← estado (UiState)
           | eventos (sealed interface)
ViewModel  ← Hilt @Inject
           |
Repository
           |
Room DB    Retrofit
(offline)  (API REST)
```

- Cada feature: `Screen` + `ViewModel` + `UiState` + `Event`
- Estado reactivo: `Flow` + `mutableStateOf`
- Navegación type-safe con `@Serializable`

App Android

Funcionalidades y pantallas

Autenticación



9:41


EcoQuest

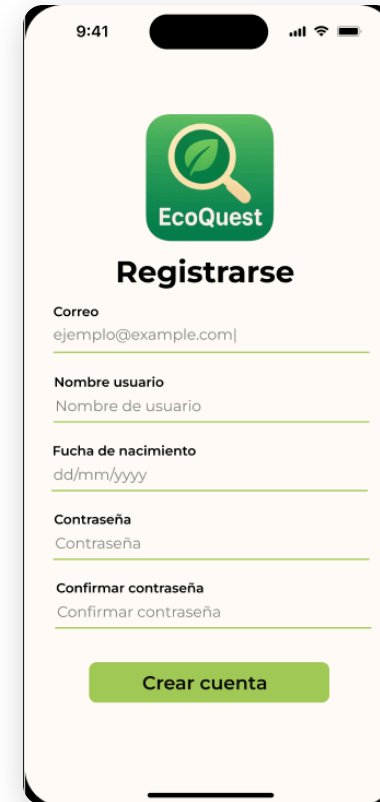
Iniciar sesión

Correo
ejemplo@example.com|

Contraseña
contraseña

Iniciar sesion

Inicio de sesión con correo y contraseña



9:41


EcoQuest

Registrarse

Correo
ejemplo@example.com|

Nombre usuario
Nombre de usuario

Fecha de nacimiento
dd/mm/yyyy

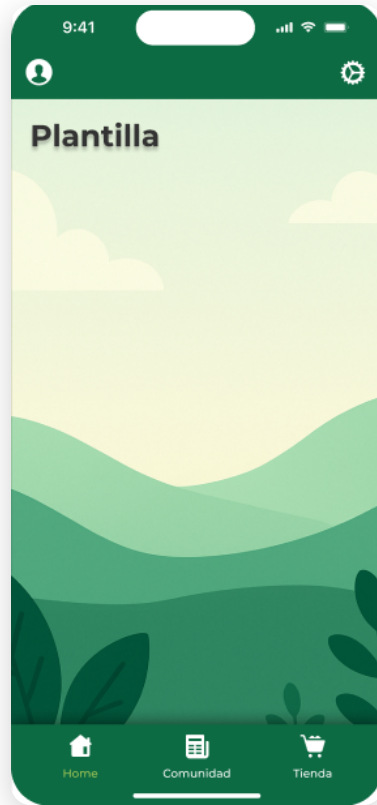
Contraseña
Contraseña

Confirmar contraseña
Confirmar contraseña

Crear cuenta

Registro con validación de campos

Pantalla principal y navegación



Home con navegación inferior: Home · Comunidad · Tienda

Estructura de navegación

- Barra inferior fija con 3 destinos principales
- Icono de perfil en la esquina superior izquierda
- Icono de ajustes en la esquina superior derecha
- Fondo generativo con paisaje en Compose Canvas

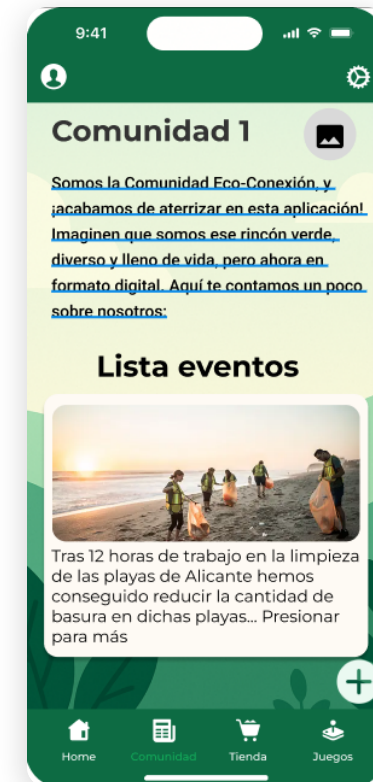
Canvas personalizado

El paisaje de colinas se dibuja con `DrawScope` y trazados cúbicos (`cubicTo`) — sin imágenes, solo código Kotlin.

Comunidades

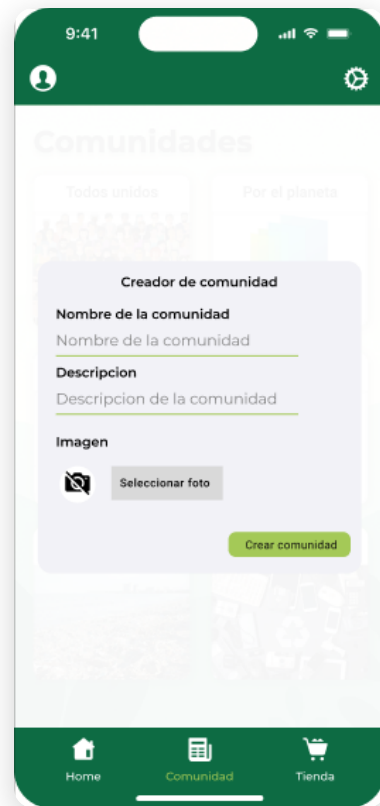


Lista de comunidades disponibles

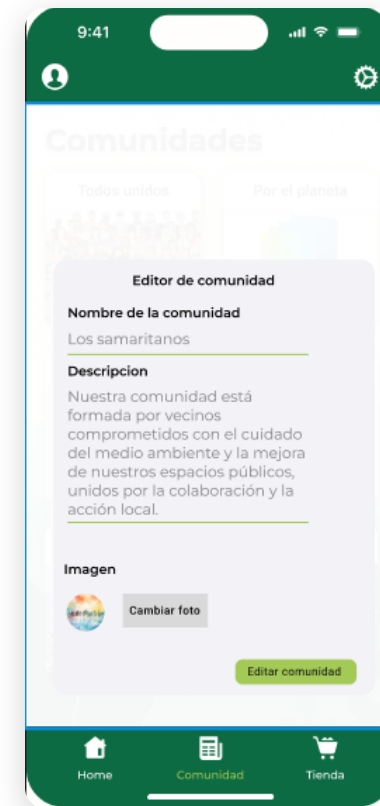


Detalle de comunidad con lista de eventos

Crear y editar comunidad



Diálogo: crear nueva comunidad

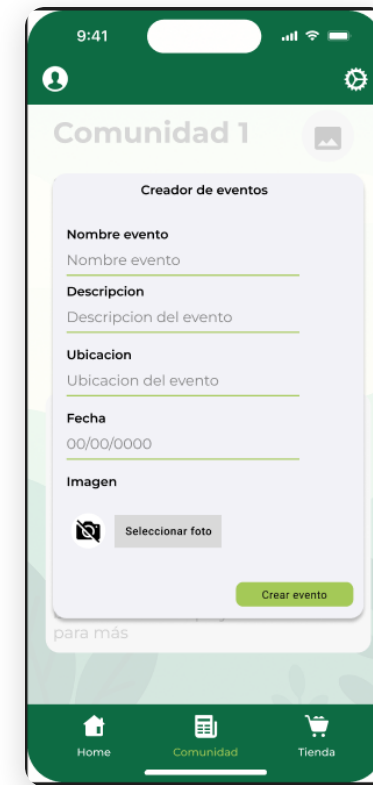


Diálogo: editar comunidad existente

Eventos

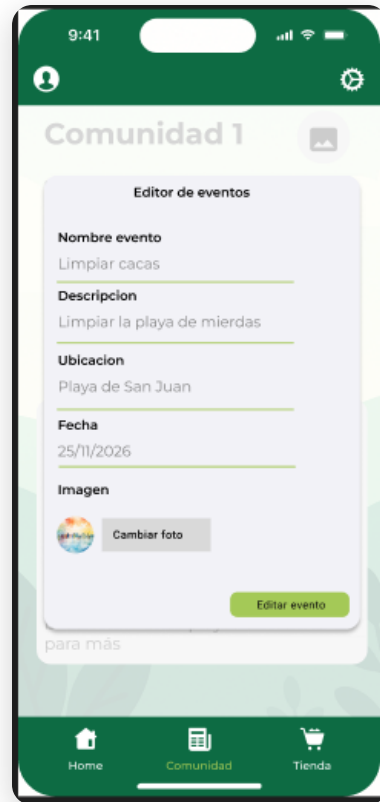


Feed de eventos con búsqueda en tiempo real



Formulario de creación de evento

Editar eventos



Editor de evento con datos precargados

Tipos de evento

Evento Comunitario

Actividades organizadas por la comunidad

Noticia

Información relevante para los miembros

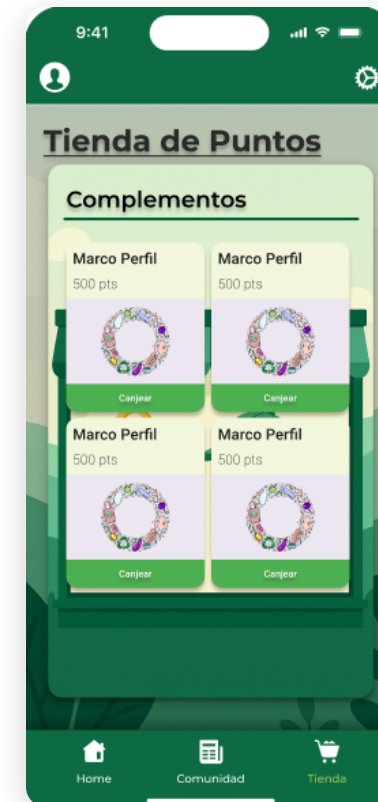
Urgente

Acciones que requieren respuesta inmediata

Tienda de Puntos

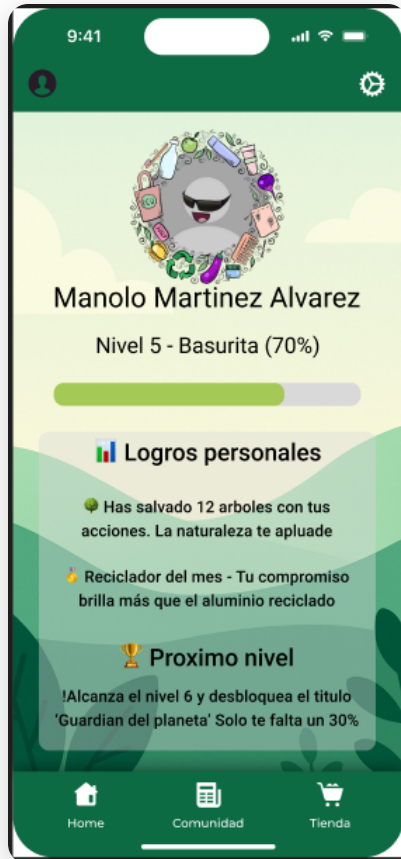


Puntos disponibles y producto destacado



Catálogo de complementos canjeables

Perfil de usuario



Perfil con nivel, logros y progreso al siguiente nivel

Gamificación del perfil

- Sistema de niveles (ej. Nivel 5 - Basurita)
- Barra de progreso hacia el siguiente nivel
- Logros personales por acciones ecológicas
- Título desbloqueado al subir de nivel

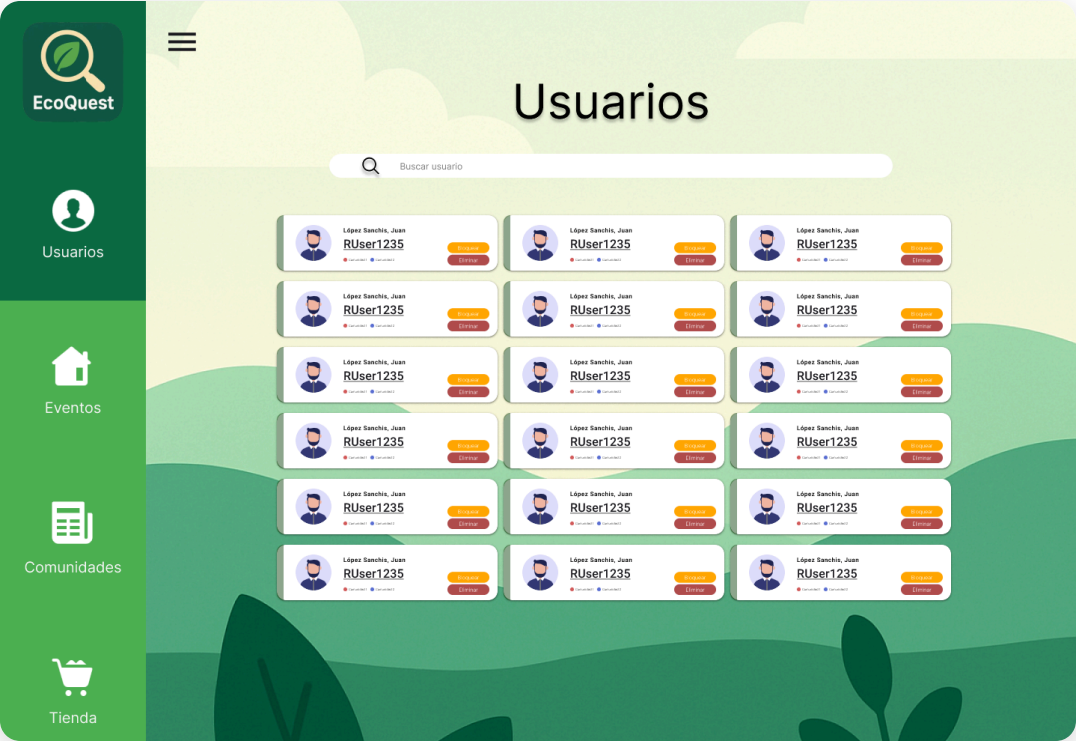
Ejemplo de logro

"Has salvado 12 árboles con tus acciones. La naturaleza te aplaude."

Panel de Administración

WPF · C# · .NET · MVVM

Gestión de usuarios

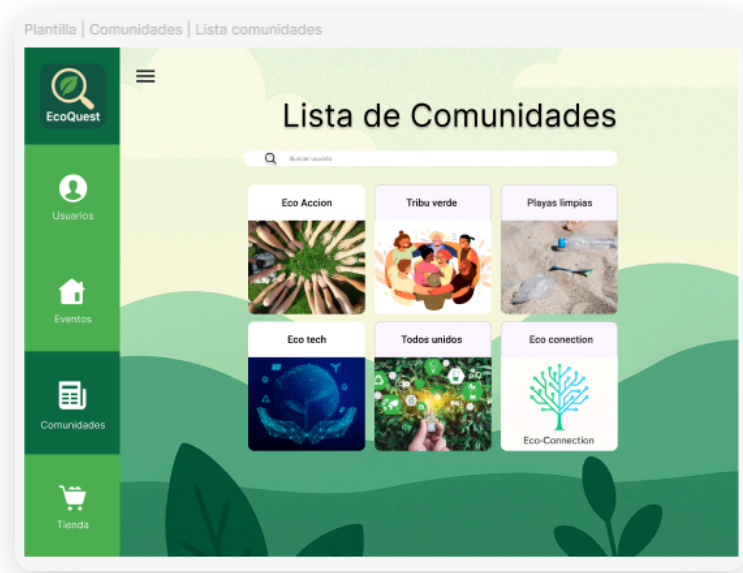


Lista de todos los usuarios registrados



Diálogo de bloqueo con causa y duración

Gestión de comunidades



Lista de comunidades activas

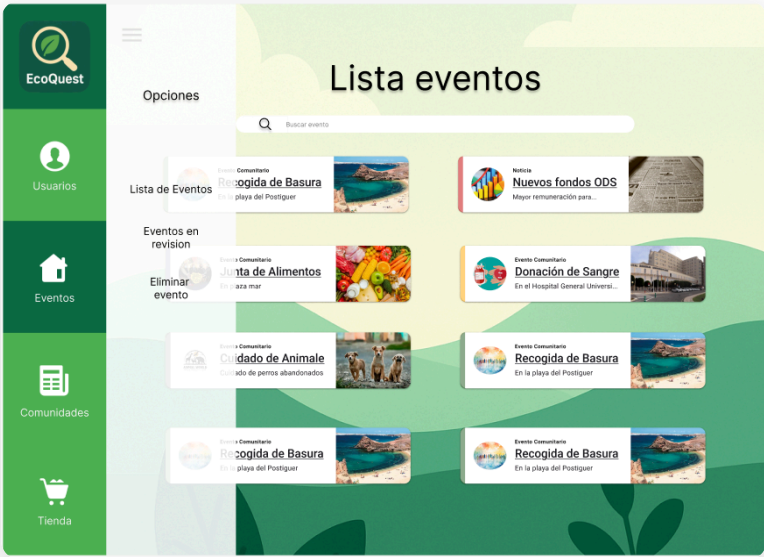


Comunidades pendientes de aprobación

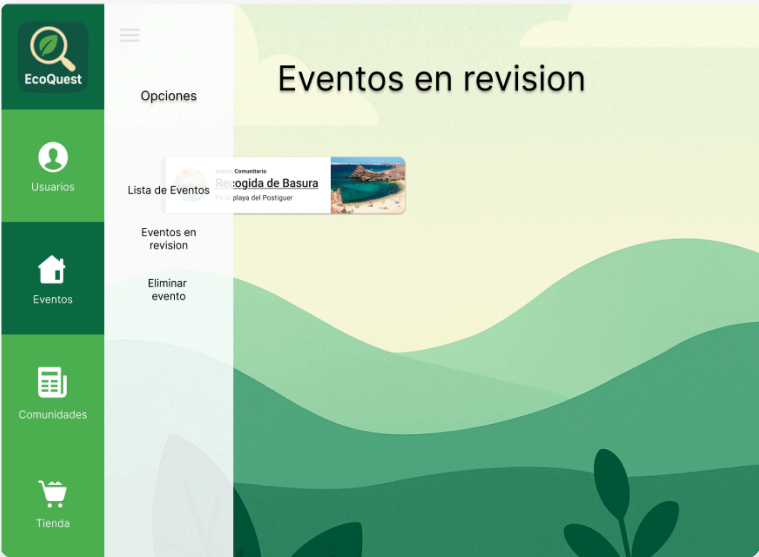


Detalle: aceptar o denegar comunidad

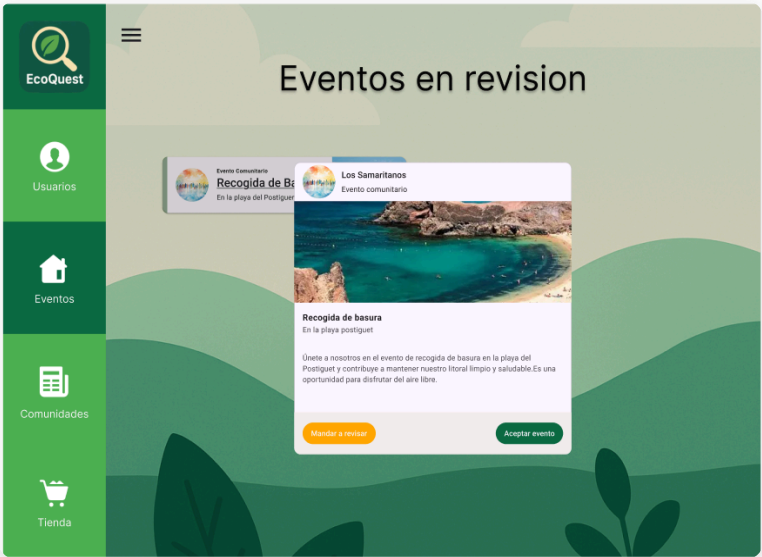
Gestión de eventos



Lista de todos los eventos

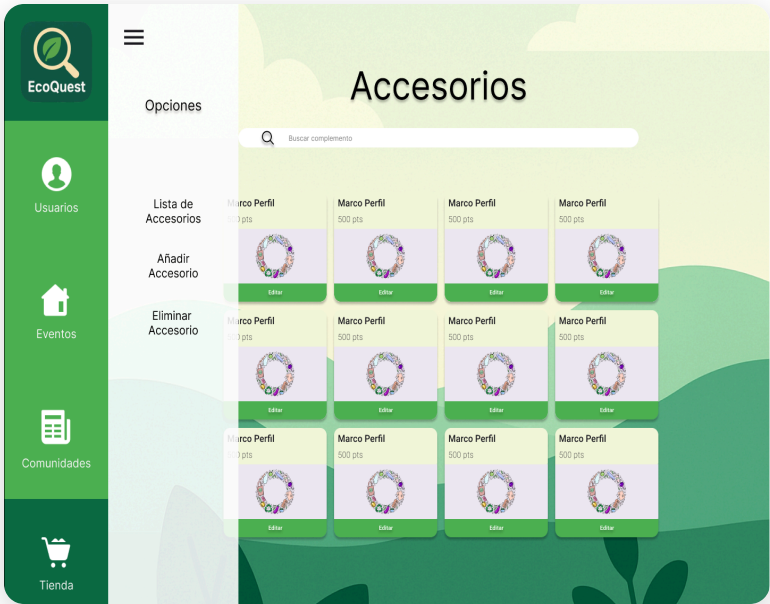


Eventos pendientes de revisión

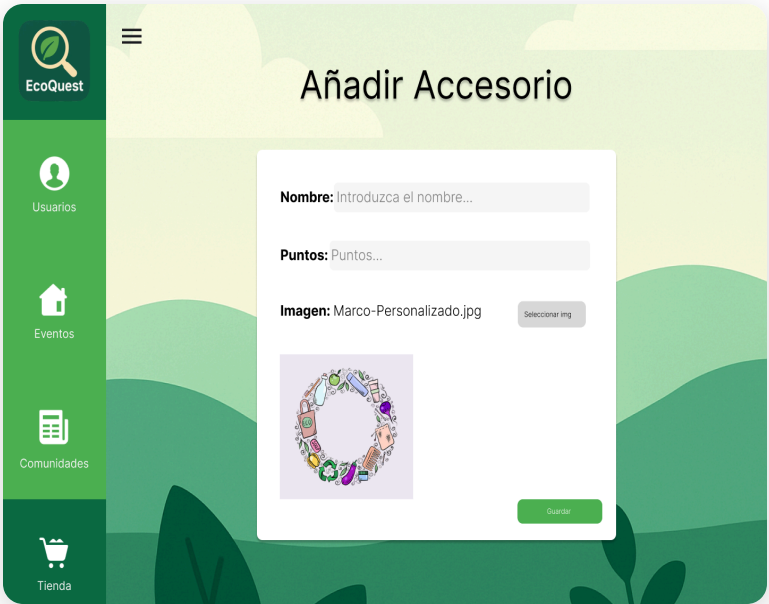


Detalle: aprobar o rechazar evento

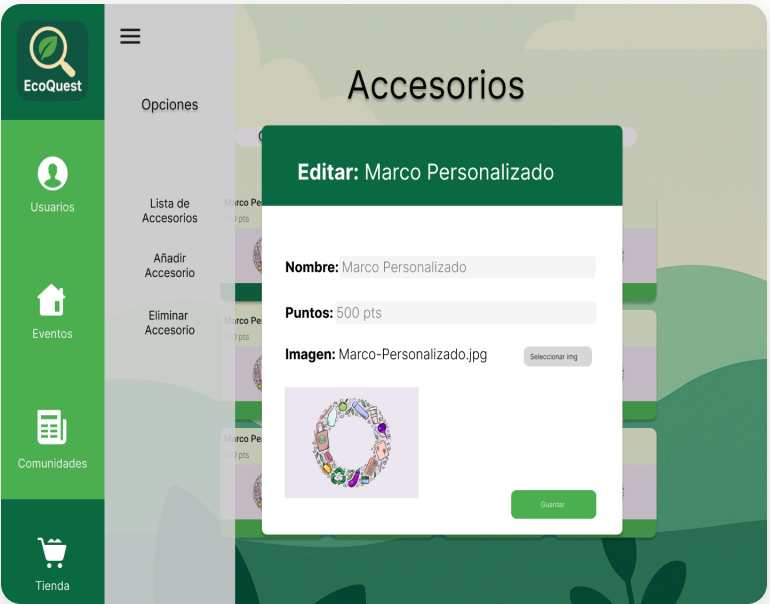
Gestión de accesorios (Tienda)



Catálogo de accesorios



Añadir accesorio nuevo



Editar accesorio existente

Características técnicas destacables

Estado reactivo — Flow + Compose

```
// ViewModel: un único estado observable
var state by mutableStateOf(EventosUiState())
    private set

init {
    viewModelScope.launch {
        // Flow de Room: la UI se actualiza sola
        // cuando cambia la base de datos
        repo.getAll().collect { lista ->
            state = state.copy(eventos = lista)
        }
    }
}
```

Inyección de dependencias — Hilt

```
@HiltViewModel
class EventosViewModel @Inject constructor(
    private val repo: EventoRepository
) : ViewModel()
```

Autenticación JWT — Retrofit interceptor

```
class AuthInterceptor(private val token: String)
    : Interceptor {
    override fun intercept(chain: Chain) =
        chain.proceed(
            chain.request().newBuilder()
                .header("Authorization", "Bearer $token")
                .build()
        )
}
```

Room — datos offline + seed idempotente

```
// IGNORE: no sobrescribe si ya existe
@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insertAllIfAbsent(
    eventos: List<EventoEntity>
)
```

Conclusión

Lo que hemos construido

- App Android con 10+ pantallas funcionales
- API REST con autenticación JWT
- Panel WPF con gestión completa
- Las 3 plataformas conectadas en tiempo real

Posibles ampliaciones

- Geolocalización de eventos en mapa
- Retos personalizados con IA
- Notificaciones push
- Versión web responsive

El equipo — Equipo Prompt

Puntos fuertes

Reparto claro por plataforma. Buen uso de Git para integrar el trabajo de todo el equipo.

Mayor reto

Coordinar 3 plataformas distintas sobre una sola API requirió definir bien los contratos desde el principio.

Mayor aprendizaje

Jetpack Compose y MVVM en un proyecto real. También la integración completa backend, móvil y escritorio.



Gracias

Esta ha sido nuestra propuesta — EcoQuest

Equipo Prompt · IES Doctor Balmis · DAM 2025–2026

github.com/Prompt-pi2damiesbalmis/PROYECTO12